



ALTAY YAZILIM SAVUNMA ENDÜSTRİYEL SANAYİ

FULL STACK YAZILIM GELİŞTİRME STAJ RAPORU

Proje Adı: ALTAY PUBS

Hazırlayan: Burak ÖZTEKİN
Danışman: Mustafa TÜTÜNCÜ

Eylül 2026 | Ankara

İçindekiler

1. Giriş.....	3
2. Donanım ve Geliştirme Ortamı Bilgileri	3
3. Yazılım ve Sürüm Bilgileri	4
3.1 Teknolojilerin Kavramsal Rollerini.....	4
4. Proje Oluşturma ve Kurulum	5
5. Veri Tabanı (Pubs) ve Veri Modeli	5
5.1 Kullanılan Tablolar ve İlişkiler	5
6. Uygulama Mimarisi	6
7. Backend Geliştirme.....	7
7.1 Veri Modeli ve DbContext (Database-First).....	7
7.2 Servis ve API Katmanı.....	7
7.3 Hata Yönetimi ve Veri Doğrulaması.....	7
8. Frontend Geliştirme	8
8.1 Arayüz Tasarımı ve Component Mimarisi.....	8
8.2 API İletişimi ve Asenkron Veri Yönetimi.....	8
8.3 Hata Yönetimi ve Gelişmiş Form Doğrulama (Frontend Validation)	8
9. Erişim Güvenliği	9
9.1 JWT Tabanlı Kimlik Doğrulama.....	9
9.2 Rol Bazlı Yetkilendirme.....	9
9.3 PBKDF2 ile Parola Şifreleme (Hashing & Salting).....	9
9.4 Kullanıcı Yönetimi Modülü	10
10. Karşılaşılan Sorunlar ve Çözümler	10
11. Sonuç.....	11

1. Giriş

Projenin amacı, pubs veri tabanı altyapısı kullanılarak tasarlanan yazılım prensiplerine ve güvenli kodlama standartlarına uygun olarak geliştirilmiş full stack web uygulamasının (PubsApp) mimari yapısını, geliştirme süreçlerini ve uygulanan güvenlik protokollerini detaylandırmaktadır. Yayıncılık sektöründeki kitaplar (titles), yazarlar (authors) ve bunlar arasındaki ilişkilerin (TitleAuthors) yönetimini sağlamak üzere tutarlı ve güvenli bir sistem inşa etmektir.

Uygulamanın ön yüzü (frontend) angular framework'üyle bileşen tabanlı olarak tasarlanmış, arka uç (backend) hizmetleri ise .NET Core Web API teknolojisiyle uygulanmıştır. Uygulamanın API katmanında geleneksel controller mimari yapı sunan Minimal API (authors modülü için) kurgusu hibrit bir yaklaşımla kullanılmıştır. Veri tabanı işlemleri entity framework core aracılığıyla yürütülmüş olup, sistemin genel mimarisi clean architecture prensiplerinden fikir alarak veri, servis ve sunum katmanlarını birbirinden izole etmiştir. Geliştirilen sistemde; hatalı ID formatları (örneğin 2 harf + 4 rakam kuralı) ve eksi bakiye/satış değerleri gibi mantıksal hataları önlemek amacıyla, SQL veri tabanı kısıtlamaları (constraints), C# data annotations ve angular form doğrulama mekanizmaları senkronize edilerek üç katmanlı bir veri koruma duvarı oluşturulmuştur.

Sistem güvenliği bağlamında; yetkisiz erişimleri engellemek için ön yüzde Angular Auth Guards (route korumaları), arka uçta ise JWT (JSON Web Token) tabanlı kimlik doğrulama mekanizması entegre edilmiştir. Ayrıca KVKK ve güvenlik standartları gereğince, sisteme ait kullanıcı parolaları veri tabanında düz metin olarak saklanmak yerine, PBKDF2 algoritması ve rastgele tuzlama (salt) yöntemleri kullanılarak geri döndürülemez özetler (hash) halinde şifrlenerek muhafaza altına alınmıştır.

2. Donanım ve Geliştirme Ortamı Bilgileri

Projenin geliştirme, derleme ve test süreçleri yerel bir çalışma ortamında gerçekleştirilmiştir. Angular geliştirme sunucusu, .NET Web API ve SQL Server servislerinin eşzamanlı çalışabilmesi için kullanılan sistemin donanım özellikleri Tablo 1'de gösterilmiştir.

Özellik	Detay
İşletim Sistemi	Windows 10 Pro, Sürüm 22H2
İşlemci (CPU)	Intel(R) Xeon(R) CPU E3-1246 v3 3.50GHz
Sistem Belleği (RAM)	16 GB DDR3
Grafik Kartı (GPU)	NVIDIA Quadro K620 (2 GB) & Intel(R) HD Graphics P4600/P4700
Depolama	HP SSD S750 512GB
Geliştirme Ortamı (IDE)	Visual Studio 2026

Tablo 1

3. Yazılım ve Sürüm Bilgileri

Uygulamanın frontend ve backend bağımlılıklarının çakışmaması ve projenin farklı ortamlarda hatasız olabilmesi için kullanılan temel araçlar Tablo 2'de verilmiştir.

Yazılım	Sürüm	Kullanım Amacı
Node.js & NPM	v22.x / v11.3.x	Angular CLI ve ilişkili frontend paketlerinin kurulumu ve yönetimi.
Angular CLI	19.x	Frontend uygulamasının (SPA) oluşturulması ve derlenmesi.
.NET SDK	8.0	Backend API projesinin geliştirme ve çalışma platformu.
Entity Framework Core	8.x	SQL Server üzerindeki Pubs veri tabanına ORM erişimi.
SQL Server	2019	Veri tabanının yerel geliştirme ortamında barındırılması.

Tablo 2

3.1 Teknolojilerin Kavramsal Roller

Kullanılan mimari araçların PubsApp projesi içerisindeki teknik görevleri Tablo 3'te özetlenmiştir.

Teknoloji	Sistemdeki Görevi
Angular	Kullanıcı arayüzünü bileşen (component) tabanlı olarak oluşturan SPA framework'ü.
.NET Web API	İstemciden gelen HTTP isteklerini karşılayan, doğrulama kurallarını işleten ve veri tabanı ile haberleşen backend sunucusu.
EF Core (Database-First)	Mevcut Pubs veri tabanı şemasından otomatik C# entity modelleri üreten tersine mühendislik aracı.
Minimal API	Authors modülü için controller sınıflarına ihtiyaç duymadan doğrudan yönlendirme sağlayan hafif API mimarisi.
JWT & PBKDF2	Kimlik doğrulama, rol bazlı oturum yönetimi ve veri tabanındaki kullanıcı parolalarının geri döndürülemez şekilde (salt ile) şifrelenmesi.

Tablo 3

4. Proje Oluşturma ve Kurulum

Visual Studio 2026 IDE'si üzerinde "Angular and ASP.NET Core" proje şablonu kullanılarak oluşturuldu. Bunun sayesinde, Angular tabanlı frontend (PubsApp.Client) ve .NET Web API tabanlı backend (PubsApp.Server) tek bir çözüm altında entegre edilmiştir. Geliştirme sürecine başlanmadan önce, projenin ihtiyaç duyduğu çalışma zamanı ve CLI (Command Line Interface) araçlarının ortamdaki sürümleri terminal üzerinden aşağıdaki komutlarla doğrulanmıştır:

- `node -v` ve `npm -v`: Angular bağımlılıkları için Node.js ve paket yöneticisi sürüm kontrolü.
- `ng version`: Frontend framework'ü olan Angular CLI sürüm kontrolü.
- `dotnet --version`: Backend API projesi için .NET SDK kontrolü.
- `dotnet ef --version`: Veri tabanı modelleme işlemleri için Entity Framework Core araçlarının kontrolü.

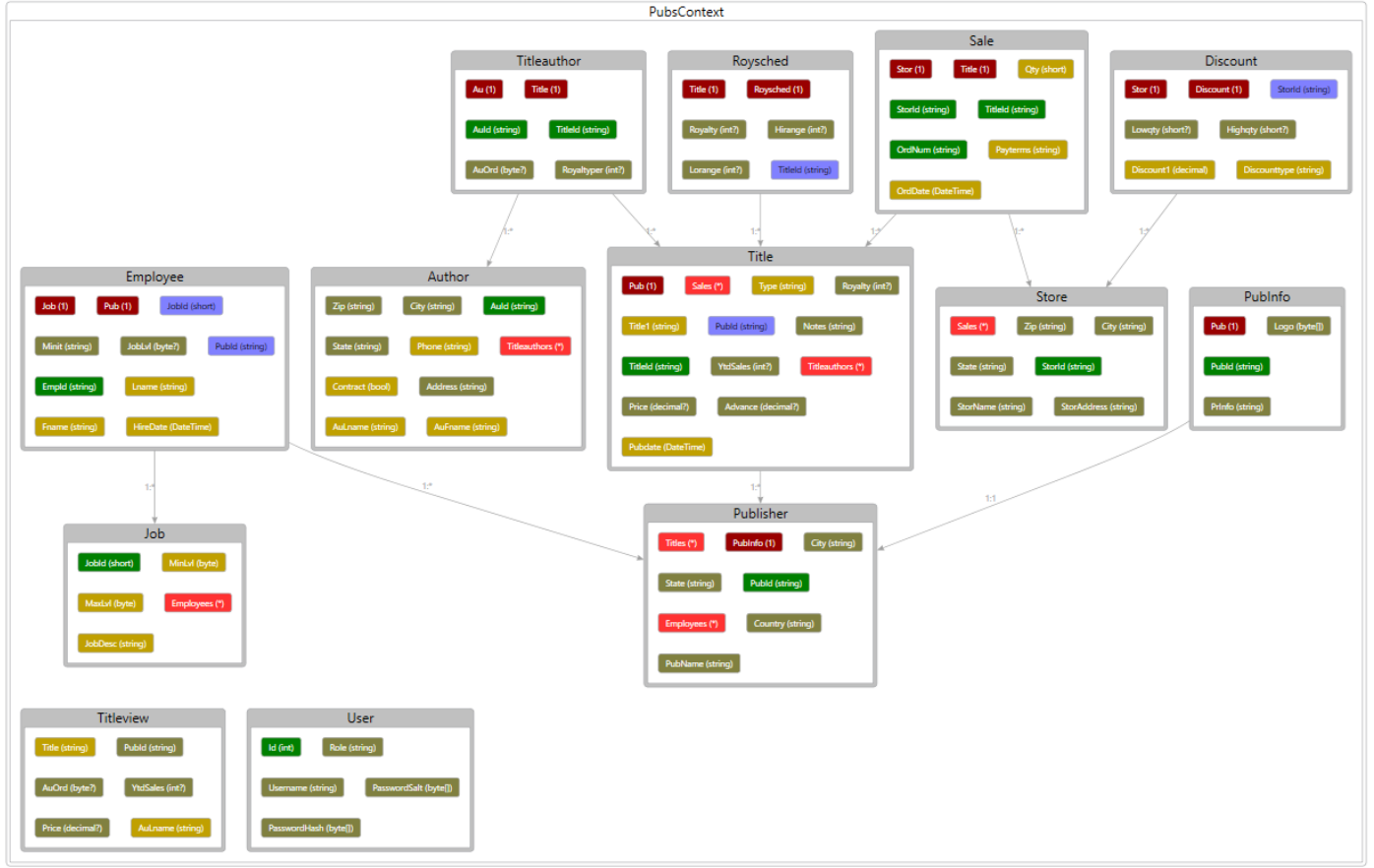
5. Veri Tabanı (Pubs) ve Veri Modeli

SQL Server üzerinde barındırılan klasik pubs veri tabanı kullanılmıştır. Veri tabanı şeması, Entity Framework Core Power Tools (Database First yaklaşımı) aracılığıyla tersine mühendislik işleminden geçirilerek C# entity sınıflarına dönüştürülmüş ve PubsContext aracılığıyla sisteme bağlanmıştır.

5.1 Kullanılan Tablolar ve İlişkiler

Sistemin amaca yönelik, güvenli ve tutarlı çalışabilmesi için veri tabanındaki aşağıdaki tablolar aktif olarak kullanılmış ve yönetilmiştir:

- **Titles:** Kitaplara ait benzersiz kimlik (ID), isim, tür, fiyat ve yıllık satış (YTD Sales) verilerini tutar. Veri bütünlüğünü sağlamak adına, ID alanında "2 harf + 4 rakam" Regex kısıtlaması ve fiyat/satış alanlarında negatif değer girilememe kısıtlaması bulunmaktadır.
- **Authors:** Yazarlara ait kimlik, isim, iletişim ve adres bilgilerini barındırır.
- **TitleAuthors:** Titles ve Authors tabloları arasındaki çoktan çoğa (M:N) ilişkiyi kuran köprü (junction) tablosudur. Bir kitabın birden fazla yazarı olabileceği gibi, bir yazarın da birden fazla kitabı olabileceği mantığını veri tabanında haritalandırır.
- **Users:** Sistemin erişim güvenliğini sağlayan tablodur. Kimlik doğrulama işlemleri için kullanıcı adı, yetkilendirme bilgisi ve güvenliği artırmak amacıyla PBKDF2 algoritmasıyla oluşturulmuş parola özetlerini (Hash ve Salt) saklar.



Görsel 1: ER Diyagram EF Core Power Tools

6. Uygulama Mimarisi

PubsApp projesi, modern web geliştirme standartlarına uygun olarak İstemci-Sunucu (Client-Server) mimarisinde tasarlanmıştır. Sistem, Clean Architecture prensiplerinden esinlenerek sorumlulukların birbirinden ayrıldığı bir yapı üzerine inşa edilmiştir.

- **Ön Uç (Client):** Kullanıcı arayüzü ve form denetimleri Angular framework ile SPA (Single Page Application) olarak geliştirilmiştir.
- **Arka Uç (Server):** HTTP isteklerini karşılayan, iş kurallarını işleten ve veri tabanı erişimini sağlayan katman .NET Core Web API ile oluşturulmuştur.
- **Katmanlı İzolasyon:** Backend projesi içerisinde veri modelleri (Models), veritabanı bağlamı (Data/Context), API yönlendirmeleri (Controllers ve Endpoints) birbirinden ayrı klasör ve sınıflarda organize edilerek kodun test edilebilirliği artırılmıştır.

7. Backend Geliştirme

Backend geliştirme sürecinde performans, güvenlik ve kod okunabilirliği esnek bir API altyapısı kurulmuştur.

7.1 Veri Modeli ve DbContext (Database-First)

Mevcut Pubs SQL veri tabanı şeması, Entity Framework Core kullanılarak Database-First yaklaşımıyla koda dönüştürülmüştür. Tersine mühendislik (scaffolding) işlemi sayesinde veritabanındaki tablolar Title, Author ve Titleauthor isimli C# entity sınıflarına haritalanmıştır. Veri tabanı ile uygulama arasındaki iletişim, Bağımlılık Enjeksiyonu (Dependency Injection) yöntemiyle Program.cs dosyasına kaydedilen PubsContext sınıfı üzerinden sağlanmıştır.

7.2 Servis ve API Katmanı

Projenin HTTP uç noktalarında (endpoints) tek bir mimariye bağlı kalınmamış, ihtiyaca yönelik hibrit bir API kurgusu tasarlanmıştır:

- **Controller Tabanlı Mimari:** Karmaşık iş kuralları içeren Titles ve TitleAuthors modülleri için geleneksel Controller sınıfları (TitlesController) kullanılmıştır. Gelen istekler bu katmanda karşılanıp asenkron (async/await) EF Core metotları ile işlenmiştir.
- **Minimal API Mimarisi:** Daha hafif ve doğrudan veri iletişimi gerektiren Authors modülü için .NET'in modern Minimal API yapısı tercih edilmiştir. Bu kapsamda AuthorEndpoints.cs sınıfı oluşturulmuş ve Program.cs dosyasında app.MapAuthorEndpoints() genişletme metoduyla sisteme dahil edilmiştir.

7.3 Hata Yönetimi ve Veri Doğrulaması

Veri bütünlüğünü sağlamak ve hatalı girişleri veritabanına ulaşmadan API katmanında engellemek için C# **Data Annotations** (Veri Nitelikleri) kullanılmıştır:

- **Regex Format Koruması:** TitleId alanı için [RegularExpression(@"^[a-zA-Z]{2}\d{4}\$")] niteliği kullanılarak ID'lerin zorunlu olarak "2 harf ve 4 rakam" formatında (örn: BU1032) gelmesi garanti altına alınmıştır.
- **Mantıksal Sınır Koruması:** Fiyat (Price) ve Yıllık Satış (YtdSales) alanları için [Range(0, double.MaxValue)] niteliği tanımlanarak eksi bakiye girişleri engellenmiştir.
- Kurallara uymayan HTTP isteklerinde, backend otomatik olarak hata fırlatarak istemciye İngilizce ve anlamlı hata mesajları ("Price cannot be negative" vb.) dönmektedir.

8. Frontend Geliştirme

Uygulamanın kullanıcı arayüzü, modüler ve yüksek performanslı bir kullanıcı deneyimi sunmak amacıyla Angular framework'ü kullanılarak SPA (Single Page Application) mimarisinde geliştirilmiştir.

8.1 Arayüz Tasarımı ve Component Mimarisi

Sistemdeki temel veri yapıları olan Titles, Authors ve aralarındaki ilişkiler için birbirinden bağımsız Angular bileşenleri oluşturulmuştur. Tasarım aşamasında standart HTML5/CSS3 yapılarına ek olarak form elemanları kullanılmıştır. Veri listeleme (data grid) ve form sayfaları arasında asenkron geçişler sağlanarak sayfa yenilenmesi önlenmiştir.

8.2 API İletişimi ve Asenkron Veri Yönetimi

Frontend ile .NET Web API arka ucu arasındaki iletişim HttpClient modülü üzerinden sağlanmaktadır. Her bir bileşen kendi servisine bağlanarak CRUD (Create, Read, Update, Delete) işlemlerini Observable akışları üzerinden asenkron olarak gerçekleştirir. API'den gelen veriler (örneğin kitap listeleri veya yazar detayları), Angular'ın veri bağlama (data binding) özellikleri sayesinde arayüze yansıtılmaktadır.

8.3 Hata Yönetimi ve Gelişmiş Form Doğrulama (Frontend Validation)

Arka uçtaki (backend) güvenlik duvarının bir benzeri, Angular formlarında (frontend) çift yönlü bir koruma mekanizması oluşturulmuştur:

- **Regex ve HTML5 Pattern Kontrolü:** HTML formlarındaki input alanlarına pattern (örn: pattern="^[a-zA-Z]{2}[0-9]{4}\$") özellikler eklenerek hatalar anında tespit edilmektedir.
- **TypeScript Validasyonu:** titles.component.ts gibi bileşenlerin TypeScript tarafında, veriler API'ye gönderilmeden hemen önce test() metotlarıyla Regex üzerinden tekrar denetlenmekte; ayrıca eksi fiyat veya küsuratlı satış rakamı gibi mantıksal hatalar if (!this.currentTitle.price < 0) gibi bloklarla durdurulmaktadır.
- **Kullanıcı Deneyimi (UX):** Ekranda kullanıcıyı yönlendiren "Title ID must be 2 letters followed by 4 digits" gibi spesifik, kırmızı renkli hata mesajları gösterilmekte ve sayfa otomatik olarak hatanın olduğu bölüme kaydırılmaktadır (window.scrollTo(0, 0)).

- **Dinamik Backend Hata Yakalama:** API'den dönen karmaşık array yapısındaki veri modelleme hataları, frontend tarafında .reduce ve .join gibi TypeScript veri işleme metotlarıyla düzleştirilerek, kullanıcının kolayca okuyabileceği temiz string mesajlara dönüştürülmektedir.

9. Erişim Güvenliği

Uygulamanın veri güvenliğini ve kullanıcı gizliliğini sağlamak amacıyla uçtan uca şifreleme ve yetkilendirme standartları uygulanmıştır.

9.1 JWT Tabanlı Kimlik Doğrulama

Sisteme giriş yapan kullanıcıların kimlik doğrulaması JWT altyapısı ile sağlanmaktadır. Başarılı bir giriş işleminin ardından backend, kullanıcıya ait kimlik ve rol bilgilerini barındıran, HMAC-SHA256 algoritmasıyla imzalanmış bir token üretir. Angular ön yüzü bu tokeni güvenli bir şekilde saklar ve HTTP interceptor mimarisi aracılığıyla API'ye yapılan sonraki tüm isteklerin authorization başlığına otomatik olarak ekleyerek oturum sürekliliğini sağlar.

9.2 Rol Bazlı Yetkilendirme

Sistemde veri güvenliğini kademelendirmek amacıyla "Admin" ve "User" olmak üzere rol bazlı bir hiyerarşi kurgulanmıştır.

- **Backend Kontrolü:** Hassas veri değişikliği gerektiren Ekleme (POST), Güncelleme (PUT) ve Silme (DELETE) API uç noktaları ilgili rollere göre korunarak yetkisiz erişimlere kapatılmıştır.
- **Frontend Kontrolü:** Angular tarafında Auth Guards kullanılarak yetkisiz kullanıcıların yönetim sayfalarına erişimi URL bazında engellenmiş; arayüzdeki menüler ve işlem butonları (Sil, Düzenle) aktif oturumun rolüne göre dinamik olarak yönetilmiştir.

9.3 PBKDF2 ile Parola Şifreleme (Hashing & Salting)

Kullanıcı parolaları veri tabanında düz metin (plain-text) olarak saklanmamaktadır. Parolalar, brute force saldırılarına karşı korunaklı olup PBKDF2 (Password-Based Key Derivation Function 2) algoritmasıyla şifrelenmiştir. Her bir parola için özel olarak üretilen rastgele bir "Tuz" (Salt) değeri kullanılarak eşsiz bir hash elde edilir. Giriş işlemleri sırasında, kullanıcının girdiği parola aynı tuz değeriyle tekrar matematiksel özetlemeye tabi tutulur ve veri tabanındaki güvenli veriyle karşılaştırılarak doğrulama yapılır. Sistem hiçbir aşamada asıl parolayı geri dönüştürmez veya bilmez.

9.4 Kullanıcı Yönetimi Modülü

Sistemdeki hesapların denetimini sağlamak adına yalnızca Admin yetkisine sahip profillerin erişebildiği bir kullanıcı yönetimi arayüzü tasarlanmıştır. Bu panel üzerinden sisteme yeni kullanıcı tanımlama, rol atama ve kimlik doğrulama süreçlerinin yönetimi gerçekleştirilmektedir.

10. Karşılaşılan Sorunlar ve Çözümler

Geliştirme süreci boyunca full-stack mimarının doğasından kaynaklanan çeşitli teknik zorluklarla karşılaşmış ve çözüme ulaşmıştır.

Sorun Kategorisi	Açıklama	Çözüm
API Hata Mesajlarının Ön Yüzde İşlenmesi	Backend'den dönen karmaşık validasyon hataları array formatında geldiği için, başlangıçta kullanılan .flat() metodu bazı TypeScript derleme ortamlarında uyumluluk sorunu yaratmış ve arayüzde objelerin "[object Object]" şeklinde görünmesine yol açmıştır.	Hata dizilerini düzeltmek ve birleştirmek için .flat() yerine daha stabil olan .reduce((acc, val) => acc.concat(val), []) ve .join('\n') kullanılarak hata mesajları metinlere dönüştürüldü.
Minimal API Routing Eksikliği	Authors modülü için yazılan Minimal API uç noktaları (Endpoints) başlangıçta 404 (Not Found) hatası döndürmüş ve arayüz veriye ulaşamamıştır.	Standart denetleyicilerin aksine Minimal API'lerin Program.cs içerisinde manuel olarak eşlenmesi (mapping) gerektiği tespit edildi. builder.Build() komutunun hemen ardına app.MapAuthorEndpoints(); satırı eklenerek sorun çözüldü.
Veri Doğrulama Senkronizasyonu	Titles tablosundaki "2 harf + 4 rakam" kimlik kısıtlaması yalnızca veri tabanı seviyesinde bırakıldığında, hatalı girişler veri tabanına kadar ulaşp gereksiz sunucu yükü ve SQL hataları yaratmıştır.	Validasyon işlemi üç katmana yayıldı. Angular tarafında HTML5 Pattern ve Regex ile hatalı giriş engellendi. Backend'de C# Data Annotations ([RegularExpression]) kullanılarak ek bir güvenlik oluşturuldu.

11. Sonuç

Bu proje kapsamında; köklü bir ilişkisel veri tabanı yapısı olan "Pubs" üzerine, yazılım teknolojileri (Angular SPA ve .NET Core Web API) kullanılarak güvenli ve ölçeklenebilir bir full-stack yönetim paneli geliştirildi.

Proje süreci, temel veri tabanı işlemlerinin (CRUD) ötesine geçerek mühendislik pratiklerinin uygulanmasına olanak tanımıştır. Geleneksel controller mimarisi ile Minimal API yapısının aynı projede hibrit olarak nasıl kurgulanabileceği test edilmiş; veri bütünlüğü sağlamak adına SQL, C# ve TypeScript katmanlarında senkronize çalışan üç aşamalı doğrulama sistemi tasarlanmıştır.

Bunun yanı sıra, JWT tabanlı oturum yönetimi, rol bazlı erişim kısıtlamaları (Admin/User) ve PBKDF2 algoritmasıyla tuzlanmış (salted) parola şifreleme yöntemleri kullanılarak uygulamanın siber güvenlik standartları en üst seviyeye çıkarılmıştır. Sonuç olarak, veri gizliliğini koruyan, esnek katmanlı mimariye (Clean Architecture) sahip ve kullanıcı deneyimini merkeze alan kurumsal düzeyde bir yazılım ürünü elde edilmiştir.